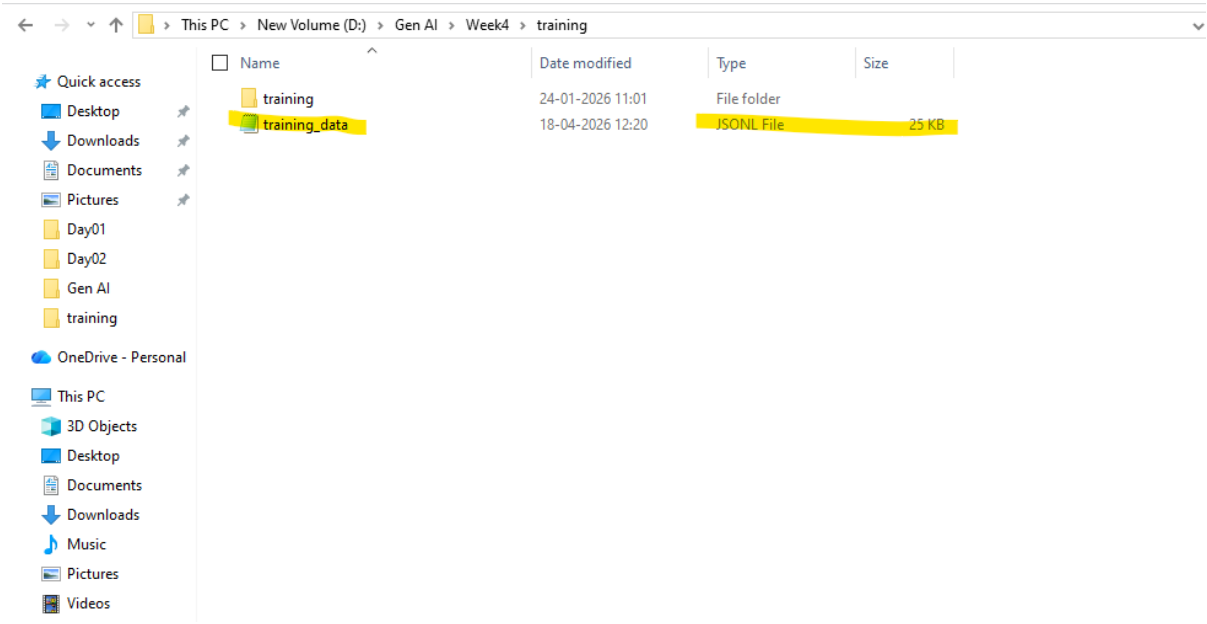
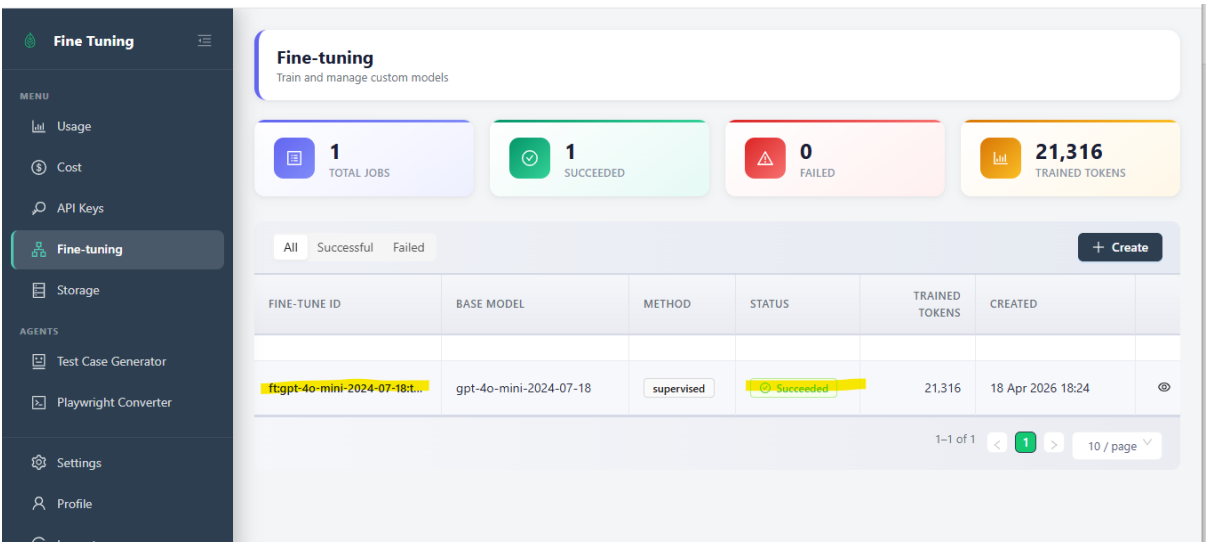


Creation of JSONL file:



Creating Fine-tuned model:



ft:gpt-4o-mini-2024-07-18:testleaf-2::DVzaiMQe

✓ Status

✓ Succeeded

🔍 Job ID

ftjob-o2sfpFF0GF4NbCAMDZYPMu7t

📘 Training Method

supervised

🧠 Base Model

gpt-4o-mini-2024-07-18

🧠 Output Model

ft:gpt-4o-mini-2024-07-18:testleaf-2::DVzaiMQe

🕒 Created At

18 Apr 2026 18:24:02

📊 Trained Tokens

21,316

🔄 Epochs

4

📋 Batch Size

1

📈 LR Multiplier

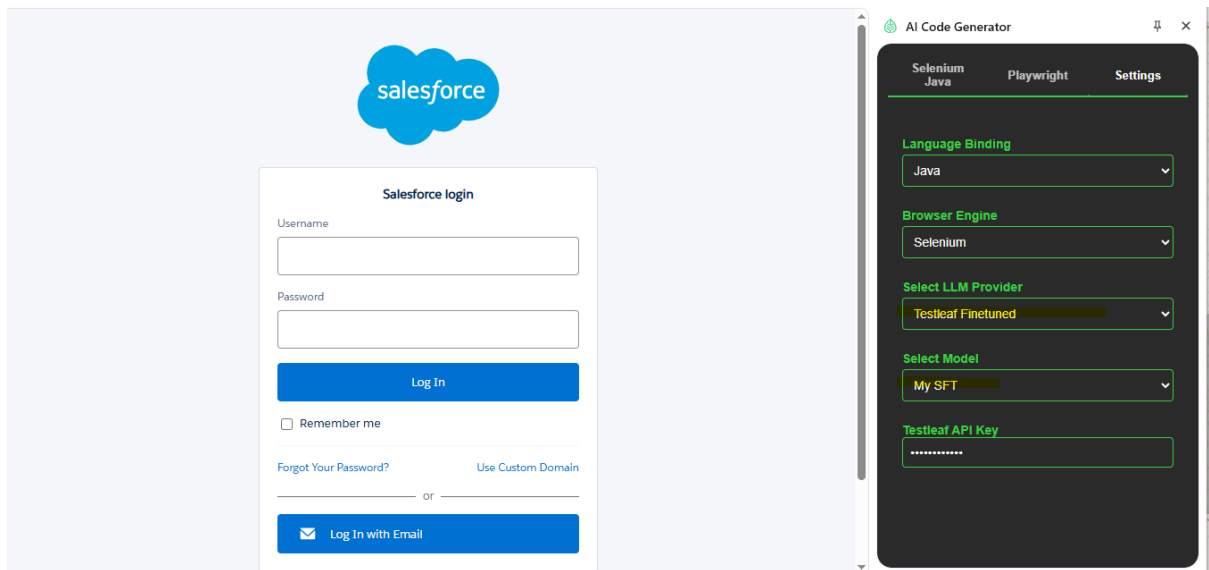
0.09

🌟 Seed

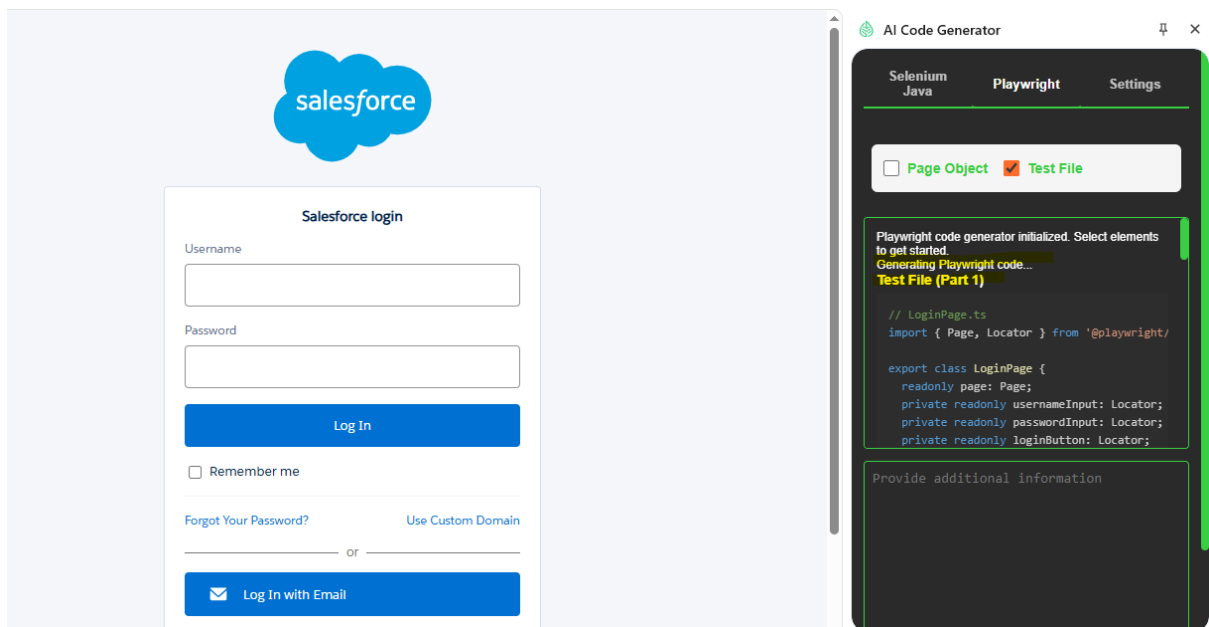
238732421

Adding the finetuned Model to the extension:

```
extension > src > scripts > JS popup.js > document.addEventListener('DOMContentLoaded') callback > updateModelOptions > models.forEach() c
5 document.addEventListener('DOMContentLoaded', async () => {
96   if (!storage) {
97     console.error('Chrome storage is not available');
98     return;
99   }
100
101   const providerSelect = document.getElementById('providerSelect');
102   const modelSelect = document.getElementById('modelSelect');
103   const groqApiKeyInput = document.getElementById('groqApiKey');
104   const openaiApiKeyInput = document.getElementById('openaiApiKey');
105   const testleafApiKeyInput = document.getElementById('testleafApiKey');
106
107   // Model options by provider
108   const modelsByProvider = {
109     groq: [
110       { value: 'meta-llama/llama-4-maverick-17b-128e-instruct', label: 'meta-llama/llama-4-maverick-17b-1' },
111       { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
112     ],
113     openai: [
114       { value: 'gpt-4o', label: 'GPT-4o' },
115       { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
116     ],
117     testleafFinetuned: [
118       { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DVzaiMQe', label: 'My SFT' },
119     ]
120   };
121
122   // Function to update model options based on selected provider
123   function updateModelOptions(provider) {
124     modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option>';
125   }
126 }
```



Generating code using the finetuned Model:



Code Generated:

Test File (Part 1)

```
// LoginPage.ts
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  private readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('input[type="submit"]#Login');
    this.errorMessage = page.locator('#chooser_error');
  }
}
```

```

}

async goto() {
  await this.page.goto('chrome-extension://pckceenoolhcohnhnijkncjogalpdk/panel.html');
}

async login(username: string, password: string) {
  await this.usernameInput.fill(username);
  await this.passwordInput.fill(password);
  await this.loginButton.click();
}

async getErrorMessage(): Promise<string | null> {
  return await this.errorMessage.textContent();
}

async isErrorMessageVisible(): Promise<boolean> {
  return await this.errorMessage.isVisible();
}
}

```

Test File (Part 2)

```

// login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test.describe('Login Feature', () => {
  let loginPage: LoginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('should successfully login with valid credentials', async () => {
    await loginPage.login('arjun.murthy@example.com', 'SecurePass@123');
    expect(await loginPage.isErrorMessageVisible()).toBeFalsy();
  });

  test('should display error with invalid credentials', async () => {
    await loginPage.login('arjun.murthy@example.com', 'wrongpassword');
    const errorMsg = await loginPage.getErrorMessage();
    expect(errorMsg).toContain('Invalid');
  });
});

```